

AdvR 12 – Metaprogramming & Expressions

Big Picture

Why Metaprogramming?

Metaprogramming = code that inspects, modifies, or generates **other code**.

R exposes this more than most languages:

- `dplyr::filter(df, x > 5)` — `x > 5` is **captured**, not evaluated
- `ggplot2::aes(x = mpg, y = hp)` — column names are **quoted**, not looked up
- `~` formulas — store expressions + environments

Three pillars: **expressions** (code as data), **quasiquote** (code templates), **evaluation** (running captured code).

`rlang::expr()` captures code without evaluating it:

```
expr(mean(x, na.rm = TRUE))
```

```
#> mean(x, na.rm = TRUE)
```

```
expr(10 + 100 + 1000)
```

```
#> 10 + 100 + 1000
```

Inside a function, use `enexpr()` to capture the **caller's** code:

```
capture_it <- function(x) enexpr(x)  
capture_it(a + b + c)
```

```
#> a + b + c
```

Captured code is called an **expression** (call, symbol, constant, or pairlist).

Modifying Expressions

Expressions behave like **lists** — use `[[` and `$`:

```
f <- expr(f(x = 1, y = 2))  
f$z <- 3  
f
```

```
#> f(x = 1, y = 2, z = 3)
```

```
f[[2]] <- NULL  
f
```

```
#> f(y = 2, z = 3)
```

The first element is the function; arguments start at position 2.

Code Is a Tree (AST)

Code has a **tree** structure — the **abstract syntax tree**:

```
lobstr::ast(f(a, "b"))
```

```
#> f  
#> a  
#> "b"  
#> ... [output truncated]
```

Nested calls create deeper trees. Operators are also calls:

```
lobstr::ast(1 + 2 * 3)
```

```
#> `+`  
#> 1  
#> `*`  
#> 2  
#> 3  
#> ... [output truncated]
```

Code Can Generate Code

`rlang::call2()` builds calls from components:

```
call2("+", 1, call2("*", 2, 3))
```

```
#> 1 + 2 * 3
```

`!!` (bang-bang) **unquotes** — inserts a stored expression into a template:

```
xx <- expr(x + x)
yy <- expr(y + y)
expr(!!xx / !!yy)
```

```
#> (x + x)/(y + y)
```

Wrap in a function for reusable code generators:

```
cv <- function(var) {
  var <- enexpr(var)
  expr(sd(!!var) / mean(!!var))
}
cv(x + y)
```

```
#> sd(x + y)/mean(x + y)
```

`base::eval()` executes an expression in an environment:

```
eval(expr(x + y), env(x = 1, y = 10))
```

```
#> [1] 11
```

```
eval(expr(x + y), env(x = 2, y = 100))
```

```
#> [1] 102
```

Two powerful uses:

- 1 **Override functions** (DSLs)
- 2 **Data masks** (refer to columns as variables)

Custom Evaluation: Functions

Bind **functions** in the evaluation environment to change behaviour:

```
string_math <- function(x) {  
  e <- env(caller_env(),  
    `+` = function(x, y) paste0(x, y),  
    `*` = function(x, y) strrep(x, y))  
  eval(enexpr(x), e)  
}  
name <- "Hadley"  
string_math("Hello " + name)
```

```
#> [1] "Hello Hadley"
```

```
string_math(("x" * 2 + "-y") * 3)
```

```
#> [1] "xx-yxx-yxx-y"
```

This is how dbplyr translates R to SQL.

`rlang::eval_tidy()` adds a **data mask** — columns act like variables:

```
df <- data.frame(x = 1:5, y = sample(5))  
eval_tidy(expr(x + y), df)
```

```
#> [1] 6 6 5 7 6
```

Wrap it into a `with()`-like function:

```
with2 <- function(df, expr) {  
  eval_tidy(enexpr(expr), df)  
}  
with2(df, x + y)
```

```
#> [1] 6 6 5 7 6
```

Problem: if `with2()` defines an internal variable `a`, it can shadow the caller's `a`.

A **quosure** bundles an expression + its environment:

```
with2 <- function(df, expr) {  
  a <- 1000  
  eval_tidy(enquo(expr), df)  
}  
df <- data.frame(x = 1:3)  
a <- 10  
with2(df, x + a)
```

```
#> [1] 11 12 13
```

`enquo()` captures the expression **and** where it was written. Result: `a` is 10 (caller's value), not 1000.

Rule: with data masks, always use `enquo()` instead of `enexpr()`.

- 1 What does `expr()` return when called inside a function? What about `enexpr()`?
- 2 Use `call2()` to construct the call `mean(1:10, na.rm = TRUE)`.

- 1 `expr()` returns the **literal code** you typed (e.g., `expr(x)` returns the symbol `x` regardless of what `x` is).
`enexpr()` captures the **caller's** expression — it “enriches” a promise into an expression.

2

```
call2("mean", 1:10, na.rm = TRUE)
```

```
#> mean(1:10, na.rm = TRUE)
```

Expressions

Abstract Syntax Trees

Expressions = ASTs. Conventions for `lobstr::ast()`:

- **Leaves** = symbols (e.g., `f`, `x`) or constants (`1`, `"y"`)
- **Branches** = call objects (function calls)

```
lobstr::ast(f(x, "y", 1))
```

```
#> f
#> x
#> "y"
#> 1
```

ASTs are **abstract** — they drop whitespace and comments:

```
ast(f(x, y) # this comment is dropped
)
```

```
#> f
#> x
#> y
#> ... [output truncated]
```

Infix Calls in the AST

Every infix operator is a **function call** in the AST:

```
lobstr::ast(y <- x * 10)
```

```
#>  `<-`  
#>  y  
#>  `*`  
#>  x  
#>  10  
#> ... [output truncated]
```

These are equivalent:

```
y <- x * 10  
`<-`(y, `*`(x, 10))
```

R prints expressions in infix form even if constructed with prefix:

```
expr(`<-`(y, `*`(x, 10)))
```

```
#> y <- x * 10
```


Four types form expressions:

Type	Example	str()	typeof()
Constant	TRUE, 1, "x"	logi/int/...	logical/...
Symbol	x, mean	symbol	symbol
Call	f(x)	language	language
Pairlist	(function args)	Dotted pair list	pairlist

Constants are **self-quoting**: `expr(TRUE)` is identical to `TRUE`.

Symbols

A **symbol** represents a name (like `x` or `mtcars`):

```
expr(x)
```

```
#> x
```

```
sym("x")           # string -> symbol
```

```
#> x
```

```
as_string(expr(x)) # symbol -> string
```

```
#> [1] "x"
```

```
is.symbol(expr(x))
```

```
#> [1] TRUE
```

Symbols are always length 1. For multiple symbols, use `rlang::syms()`.

Call Objects

A **call object** = captured function call. Behaves like a list:

```
x <- expr(read.table("important.csv", row.names = FALSE))  
x[[1]]          # function (symbol)
```

```
#> read.table
```

```
x[[2]]          # first argument
```

```
#> [1] "important.csv"
```

```
x$row.names     # named argument
```

```
#> [1] FALSE
```

```
length(x) - 1   # number of arguments
```

```
#> [1] 2
```

Call Objects: Standardise and Modify

`rlang::call_standardise()` expands partial argument names:

```
x <- expr(read.table("important.csv", row.names = FALSE))  
rlang::call_standardise(x)
```

```
#> read.table(file = "important.csv", row.names = FALSE)
```

Calls can be modified like lists:

```
x$header <- TRUE  
x
```

```
#> read.table("important.csv", row.names = FALSE, header = TRUE)
```

Constructing Calls

`rlang::call2()` builds calls from parts:

```
call2("mean", x = expr(x), na.rm = TRUE)
```

```
#> mean(x = x, na.rm = TRUE)
```

```
call2(expr(base::mean), x = expr(x), na.rm = TRUE)
```

```
#> base::mean(x = x, na.rm = TRUE)
```

```
call2("<=", expr(x), 10)
```

```
#> x <- 10
```

Parsing and Deparsing

Parsing: string $\rightarrow$ expression. **Deparsing:** expression $\rightarrow$ string.

```
x1 <- "y <- x + 10"  
x2 <- rlang::parse_expr(x1)  
is.call(x2)
```

```
#> [1] TRUE
```

```
rlang::expr_text(x2)
```

```
#> [1] "y <- x + 10"
```

For multiple expressions, use `parse_exprs()`:

```
rlang::parse_exprs("a <- 1; a + 1")
```

```
#> [[1]]
```

```
#> a <- 1
```

```
#>
```

```
#> ... [output truncated]
```

Parsing is lossy: comments and whitespace are dropped.

Operator Precedence

Infix operators have **precedence rules** — `ast()` reveals them:

```
lobstr::ast(1 + 2 * 3)
```

```
#>  `+`  
#>  1  
#>  `*`  
#> ... [output truncated]  
* binds tighter than +. Surprising: ! has low precedence:
```

```
lobstr::ast(!x %in% y)
```

```
#>  `!`  
#>  `%in%`  
#>  x  
#> ... [output truncated]
```

Use parentheses when in doubt. R has 30+ infix operators in 18 precedence groups.

Associativity

Most operators are **left-associative**: $1 + 2 + 3 = (1 + 2) + 3$:

```
lobstr::ast(1 + 2 + 3)
```

```
#>  `+`  
#>  `+`  
#>  1  
#> ... [output truncated]
```

Two exceptions — **right-associative**: exponentiation and assignment:

```
lobstr::ast(2^2^3)
```

```
#>  `^^`  
#>  2  
#>  `^^`  
#> ... [output truncated]
```

```
lobstr::ast(x <- y <- z)
```

```
#>  `<-`  
#>  x  
#>  `<-`  
#> ... [output truncated]
```


Walking ASTs: Recursive Functions

ASTs are trees → natural fit for **recursion**:

```
expr_type <- function(x) {  
  if (rlang::is_syntactic_literal(x)) "constant"  
  else if (is.symbol(x)) "symbol"  
  else if (is.call(x)) "call"  
  else if (is.pairlist(x)) "pairlist"  
  else typeof(x)  
}  
c(expr_type(expr("a")),  
  expr_type(expr(x)),  
  expr_type(expr(f(1, 2))))
```

```
#> [1] "constant" "symbol"    "call"
```

Base cases: constants, symbols. **Recursive cases:** calls, pairlists.

Example: Finding T and F

Detect logical abbreviations T/F in code:

```
switch_expr <- function(x, ...) {  
  switch(expr_type(x), ...,  
    stop("Unknown type: ", typeof(x)))  
}  
logical_abbr_rec <- function(x) {  
  switch_expr(x,  
    constant = FALSE,  
    symbol = as_string(x) %in% c("F", "T"),  
    call = , pairlist = purrr::some(x, logical_abbr_rec))  
}  
logical_abbr <- function(x) logical_abbr_rec(enexpr(x))  
c(logical_abbr(TRUE), logical_abbr(mean(x, na.rm = T)))
```

```
#> [1] FALSE TRUE
```

Pairlists: store function formals. Treat like lists:

```
f <- expr(function(x, y = 10) x + y)
typeof(f[[2]])
```

```
#> [1] "pairlist"
```

Missing arguments: empty symbol for missing defaults:

```
is_missing(missing_arg())
```

```
#> [1] TRUE
```

```
args <- f[[2]]
is_missing(args[[1]])    # x has no default
```

```
#> [1] TRUE
```

Expression vectors: returned by `parse()` and `expression()`. Use lists of expressions instead.

Exercises (Expressions)

- ❶ Which two atomic vector types can't appear in an expression? Why?
- ❷ What happens when you subset a call to remove the first element?

```
expr(read.csv("foo.csv", header = TRUE))[-1]
```

- ❸ Does -2^2 yield 4 or -4? Why?

Solutions (Expressions)

- ① **Raw** and **complex** vectors can't appear directly — you need a function call to create them (e.g., `as.raw()`, `complex()`), which makes them calls, not constants. Similarly, vectors of length > 1 require `c()`.
- ② Removing the first element shifts the first argument into the function position:

```
expr(read.csv("foo.csv", header = TRUE))[-1]
```

```
#> "foo.csv"(header = TRUE)
```

So "foo.csv" becomes the "function" — produces "foo.csv"(header = TRUE).

- ③ Returns -4. Exponentiation $\wedge$ has higher precedence than unary $-$:

```
lobstr::ast(-2^2)
```

```
#>  ~-~
```

```
#>  ~^^
```

```
#>    2
```

```
#> ... [output truncated]
```

The AST shows $-(2^2)$, not $(-2)^2$.